Wizard — v2 Architecture & Specifications

Version: 2.0
Date: March 2026
Author: Kiran Capoor
Status: Pre-build. Validated by SISU Health UK.

---

1. Problem Statement

> *"I kept losing track of my notes, investigations, meeting notes, tasks context, and had to spend hours collecting, writing then managing them across multiple sources."*

Every AI coding session starts from zero. Context lives across Jira, Notion, Krisp, and the codebase — disconnected, unstructured, and never pre-loaded before work begins. Engineers are the integration layer. Wizard removes that tax.

v1 identified gaps that v2 must solve:

- No database — raw data sent everywhere
- Token and context consumption unsustainably high (~300k tokens/session, ~100k/task)
- All steps manual, no orchestration
- Claude owns connections, orchestration, and reasoning simultaneously
- Serena invoked inconsistently by Claude rather than deterministically
- No reusability, no flexibility across integrations
- No state management in any real sense
- No way to measure or evaluate if the system is working correctly
- No PII protection before data reaches an AI model

---

2. Design Principles

1. **Claude owns reasoning and synthesis only.** Everything else is deterministic, testable, and auditable.
2. **Each layer has one responsibility.** The moment a layer encroaches on another's boundary, it becomes its own layer.
3. **Postgres is the source of truth.** All other stores are derived.
4. **Complexity must be justified by a specific observed problem,** not an anticipated one.
5. **Local-first.** No hosting, no multi-tenancy, no shared infrastructure in v2.
6. **PII never reaches Claude or Postgres in raw form.**
7. **Prefer reversible decisions.** Name irreversible ones explicitly.

3. Architecture

3.1 Dependency Flow

```
Integration → Security → Data ← Orchestration → Claude → Data
```

- **Integration** pulls raw data from external sources.
- **Security** scrubs PII before anything is stored.
- **Data** stores clean, structured data. Single source of truth.
- **Orchestration** reads from Data, controls when Claude is invoked, owns session lifecycle and workflow execution.
- **Claude** receives only prepared, normalised context. Produces structured output.
- **Claude's output** flows back into Data via: process → transform → validate → store.

3.2 Layer Responsibilities

Integration Layer

- Owns all external connections. Claude never reaches external systems directly.
- Sources: Notion MCP, Jira MCP, Krisp (meeting transcripts), Serena (code intelligence), GitHub (ADRs, branches, repos).
- Serena is invoked deterministically by Orchestration, not decided by Claude at runtime.
- Pulls raw data and passes it to the Security layer.

Security Layer

- Single responsibility: PII detection and removal.
- Sits between Integration and Data. Nothing sensitive ever reaches Postgres.
- Scrub only — no stubbing in v2. PII is removed, not replaced.
- Clinical data and codebase context pass through after PII scrubbing.
- Relevant regulation: UK GDPR, Data Protection Act 2018, NHS Data Security and Protection Toolkit (for clinical data contexts).

Data Layer

- **Postgres + pgvector** — single unified store. No separate vector database.
    - Postgres handles structured relational data with ACID guarantees.
    - pgvector handles semantic similarity search within the same database.

    - Eliminates sync complexity between two separate stores.
- **What Postgres owns:**
    - Meeting notes (action items, outline, key points)
    - Tasks (status, due date, priority, relationships to meetings and Jira, GitHub branch/repo)
    - Sessions (lifecycle, context, tasks started/completed)
    - Authentication and tokens
    - Integration configuration
    - Workflow state and output
    - Labelled examples for semantic threshold calibration
- **Encryption at rest** — required. Local-only deployment.
- Postgres is always source of truth. Vector search is derived — always consistent with Postgres.
- Vector DB sync is triggered by Orchestration, not by the Data layer itself.

Orchestration Layer

- Owns: DB syncs (Postgres → pgvector), pipelines, workflow execution, session lifecycle, queues and DLQ, audit trail, feedback loop triggers.
- Workflow definitions are hardcoded in `core/` — Orchestration executes them, not defines them.
- Workflow state and output live in Postgres.
- Does not interpret semantic content — it routes messages, it does not reason about them.
- Pre-flight contract: checks Postgres consistency and triggers any pending vector syncs before invoking Claude. Claude is never invoked on stale or inconsistent data.
- Serena is invoked deterministically here, not by Claude.

Claude (Reasoning & Synthesis)

- Receives only prepared, normalised, PII-free context from Orchestration.
- Never touches raw data, external connections, or orchestration logic.
- Produces structured output conforming to MCP contracts.
- Output pipeline: process → transform → validate → store back into Postgres.
- MCP defines the interface. Contract tests verify Claude's output matches schema on every run.

---

4. Session Architecture

Session Lifecycle

- Sessions belong in Postgres. Not cache — cache is volatile and lacks atomicity.
- Required storage properties: **reliability** (writes survive crashes) and **atomicity** (session is fully written or not at all).
- Session state = procedural state (where is the workflow?) stored in Postgres.
- Session semantic context = derived from pgvector at load time.

Task-Type Aware Context Loading

Context is loaded based on the type of work being started, not time of day:

Task Type	Context Pulled
Coding	Codebase (Serena), relevant tasks, recent ADRs
Debugging / Incident	Relevant code, recent changes, related tasks
Investigation	Meeting notes, tasks, ADRs, codebase
ADR	Architecture history, related decisions
Test generation	Codebase, task context

Session flow:

1. `wizard session start` — pulls tasks and meeting notes, lists them, asks which task to start.
2. `wizard task start` — loads task-type-specific context only.
3. Work begins.
4. `wizard task end` — generates summary, updates task status, stores output.
5. `wizard session end` — summary of session, updates Today page, routes learnings.

---

5. Semantic Threshold Calibration

Vector similarity is used to detect semantically incorrect links (e.g., a task linked to the wrong meeting). Process:

1. **Phase 1:** Manufacture labelled examples — known correct links and known wrong links.
2. **Phase 2:** Calibrate threshold against labelled set.
3. **Phase 3:** Collect real production corrections (wrong links caught in review) as new labelled examples.

4. **Phase 4:** Re-calibrate periodically against growing real-world set.

Threshold is owned and stored by the Data layer. Triggers for recalibration are owned by Orchestration. Orchestration routes the correction message — it does not interpret semantic content.

---

6. Project Structure

```
wizard/
├── mcp/           # Primary interface for AI (Claude Code, Claude Desktop)
├── cli/           # Primary interface for humans
│   ├── wizard init
│   ├── wizard integrate init <source> --token <token>
│   ├── wizard ide init --neovim / --vscode
│   └── wizard session start / task start / task end / session end
├── shared/        # Shared types, hardcoded constraints, base structs and classes
├── core/          # Business and domain logic — prompts, default hardcoded
workflows
├── orchestrator/  # Workflow execution, session lifecycle, DB sync, queues, DLQ,
audit trail
├── data/          # Postgres + pgvector, migrations, schema, query layer
├── security/      # PII detection and scrubbing
└── integrations/  # Notion, Jira, Krisp, Serena, GitHub — all external
connections
```

Key distinction:

- `core/` defines default workflows (ships with Wizard, ready to use on install).
- `orchestrator/` executes all workflows, including customised ones.
- `shared/` is types and constants — not business logic.
- `cli/` is the primary human interface and owns the setup experience.

---

7. Setup Experience

Target: any engineer at SISU Health UK running Wizard locally in under 30 minutes.

```bash
```

```
wizard init                                    # Starts Docker, installs plugins a
wizard integrate init notion --token <token>   # Configures Notion integration
wizard integrate init jira --token <token>     # Configures Jira integration
wizard integrate init krisp                    # Configures Krisp (login or MCP)
wizard ide init --neovim                       # Sets up Neovim with sidekick and
wizard ide init --claude-desktop               # Sets up Claude Desktop integratio
```

Setup is CLI-driven. No manual configuration files. No hunting for docs.

---

8. Build Sequence & Proof Criteria

Build in layers. Each step proves a contract before the next layer adds complexity.

Step 1: Data → Claude

Build: Postgres schema, pgvector setup, MCP interface, Claude plugin scaffold.
Prove: Contract test asserting Claude receives exactly what Postgres contains for a given query (e.g., get all meeting notes, get all tasks). Not just that something is returned — that it matches precisely.

Step 2: Orchestration → Data → Claude

Build: Orchestration layer, session lifecycle, basic workflow execution.
Prove: Claude is invoked by Orchestration with prepared context. Pre-flight check passes before invocation. Session state persists across a simulated crash.

Step 3: Integration → Security → Data → Orchestration → Claude

Build: First integration (Notion), Security layer (PII scrubbing), integration configuration via CLI.
Prove: Raw Notion data enters the pipeline. PII is detected and removed before Postgres. Clean data reaches Claude. Audit trail shows what was scrubbed.

Step 4: Claude Output Pipeline

Build: Output processing, transformation, validation, and storage back into Postgres.
Prove: Claude's output conforms to schema contract. Invalid output is rejected with an error, not silently stored. Correct output is retrievable from Postgres and matches what Claude produced.

Step 5: Full System

Build: Remaining integrations (Jira, Krisp, Serena, GitHub), task-type aware context loading, session flow, CLI setup commands.
Prove: Full session — `wizard session start` through `wizard session end` — runs end-to-end. PII never appears in Postgres. Context loaded is task-type specific. Output is stored and traceable to its origin.

---

9. Open Questions

Question	Status
Threshold calibration — what constitutes a semantically wrong link?	Unresolved. Requires labelled examples before threshold can be set.
UK GDPR and NHS DSP Toolkit compliance specifics for clinical data in scope	Needs legal review before clinical data enters any pipeline.
Serena MCP integration details for deterministic invocation	Needs spike.
Multi-user local deployment (multiple SISU Health engineers) — shared Postgres or separate instances?	Deferred to post-v2. Local-only for now.

---

10. What v2 Does Not Include

These are explicitly out of scope for v2:

- Hosted or cloud deployment
- Multi-tenancy
- Stubbing (PII scrubbing only, not replacement)
- Dynamic workflow definitions (hardcoded only)
- Semantic threshold auto-calibration (manual process in v2)
- Billing, licensing, or any commercial infrastructure
- Any data type beyond code and engineering context

---

11. First Customer

Tom (Tech Lead, SISU Health UK) has validated the problem and requested Wizard be made reusable for the SISU Health engineering team with security and polish.

Requirements from first customer: local-first, secure (PII protection), reusable across the team, polished setup experience.